

Cifrado Vigenère

FACULTAD DE INGENIERÍA

Criptografía

Dr. Alfonso Francisco de Abiega l'Eglise

NOMBRE DE LOS INTEGRANTES:

Ayala Hernández María Fernanda

López Hernández Miriam Amisadai

Tepal Briseño Hansel Yael

Ugartechea González Luis Antonio

GRUPO: 03

SEMESTRE: 2027-2.

1 Explicación del Cifrado Vigenère

El programa implementa un cifrado de Vigenère, un algoritmo de cifrado que utiliza una palabra clave para transformar un mensaje original en uno cifrado. Para hacerlo, a cada letra del mensaje se le asigna un valor numérico de 0 a 25 y se suma el valor correspondiente de la letra de la clave, repitiendo la clave si es necesario; el resultado se obtiene mediante una operación módulo 26, generando así una nueva letra. Para descifrar el mensaje se realiza el proceso inverso, restando el valor de la clave a cada letra cifrada.

1.1 Declaración de Variables y Validación de Argumentos

```
int main(int argc, char *argv[]){
    if(argc != 3){
        printf("Uso: %s <key> <inputfile.txt>\n", argv[0]);
        return 1;
    }
    const char *key = argv[1];
    const char *inputfile = argv[2];

    int keyLength = 0;
    while(key[keyLength] != '\0'){
        keyLength++;
    }

    if (keyLength == 0) {
        printf("La llave no puede estar vacía.\n");
        return 1;
    }
}
```

Listing 1: Validación de argumentos y cálculo de la longitud de la llave.

El programa se ejecuta desde la terminal recibiendo dos argumentos además del nombre del ejecutable: la llave (`argv[1]`) y el archivo de entrada (`argv[2]`). Si `argc` es distinto de 3, falta alguno de los dos parámetros, por lo que se imprime la forma correcta de uso y se aborta con `return 1`.

La longitud de la llave se calcula manualmente recorriendo el arreglo hasta encontrar el carácter nulo `'\0'`. Este valor (`keyLength`) se usa más adelante junto con el operador módulo para repetir la llave sobre todo el texto. Finalmente, se valida que la llave no esté vacía, ya que una llave de longitud 0 provocaría una división entre cero al calcular `i % keyLength`.

1.2 Lectura de la Llave y Apertura del Archivo

```
: FILE *file = fopen(inputfile, "r");
    if (file == NULL) {
        printf("No se pudo abrir el archivo %s\n", inputfile);
        return 1;
    }

    fseek(file, 0, SEEK_END);
    long fileSize = ftell(file);
    rewind(file);

    char *encryptedText = (char *)malloc((size_t)fileSize + 1);
    if (encryptedText == NULL) {
        printf("No hay memoria suficiente para leer el archivo.\n");
        fclose(file);
        return 1;
    }
```

Listing 2: Apertura del archivo y reserva de memoria dinámica.

El archivo se abre en modo lectura ("r") y se valida que fopen no haya regresado NULL, lo que indicaría que el archivo no existe o no se pudo abrir. Para conocer su tamaño en bytes sin leerlo por completo, se usa fseek al final del archivo, ftell para obtener la posición actual y rewind para regresar el cursor al inicio antes de comenzar la lectura. Con ese tamaño se reserva dinámicamente, se verifica que malloc no haya devuelto NULL antes de continuar, cerrando el archivo si la memoria no alcanzó.

1.3 Proceso de Cifrado

```
int encryptIndex = 0;
int decryptIndex = 0;
int position = 0;
int c;

while ((c = fgetc(file)) != EOF && position < (int)fileSize) {
    if (isalpha(c)) {
        int shift = toupper(key[encryptIndex % keyLength]) - 'A';

        if (isupper(c)) {
            c = ((c - 'A' + shift) % 26) + 'A';
        }
        else if (islower(c)) {
            c = ((c - 'a' + shift) % 26) + 'a';
        }

        encryptIndex++;
    }

    encryptedText[position] = (char)c;
    position++;
}
encryptedText[position] = '\0';
fclose(file);
```

Listing 3: Proceso de cifrado del mensaje leído del archivo.

El archivo se recorre carácter por carácter con `fgetc`, hasta llegar a EOF o hasta agotar el tamaño calculado previamente. Para cada carácter se pregunta primero si es una letra con `isalpha`; si no lo es (números, signos de puntuación, saltos de línea, espacios), el carácter se copia sin modificar al buffer y no hace avanzar el índice de la llave.

Cuando sí es una letra, el desplazamiento a aplicar se obtiene de la letra de la llave que le corresponde en ese momento, mediante `key[encryptIndex % keyLength] - 'A'`. El operador módulo es lo que permite que, si la llave es más corta que el texto, ésta se repita cíclicamente sobre las letras del mensaje. El uso de `toupper` sobre la letra de la llave garantiza que el desplazamiento se calcule igual sin importar si la llave se escribió en mayúsculas o minúsculas.

Una vez obtenido `shift`, se aplica la fórmula modular clásica de sustitución, distinguiendo si la letra del texto es mayúscula o minúscula para conservar su formato original ($((c - 'A' + \text{shift}) \% 26 + 'A')$ o su equivalente con 'a'). Solo después de cifrar una letra se incrementa `encryptIndex`, lo que sincroniza el avance de la llave únicamente con los caracteres alfabéticos del mensaje.

1.4 Proceso de Descifrado

```
i = 0;
while (encryptedText[i] != '\0') {
    c = encryptedText[i];

    if (isalpha(c)) {
        int shift = toupper(key[decryptIndex % keyLength]) - 'A';

        if (isupper(c)) {
            c = ((c - 'A' - shift + 26) % 26) + 'A';
        }
        else if (islower(c)) {
            c = ((c - 'a' - shift + 26) % 26) + 'a';
        }

        decryptIndex++;
    }

    putchar(c);
    i++;
}
```

Listing 4: Proceso de descifrado del mensaje.

El descifrado no vuelve a leer el archivo original: parte del propio `encryptedText` que ya está en memoria. Por cada carácter alfabético se calcula el desplazamiento de la misma forma que en el cifrado, pero usando un contador independiente, `decryptIndex`, que también solo avanza con letras. Esto mantiene sincronizada la llave entre el cifrado y el descifrado, sin importar cuántos espacios o signos de puntuación haya de por medio.

La operación es la inversa a la de cifrado: se resta el shift en lugar de sumarlo. Como en C el operador % puede devolver un valor negativo cuando el dividendo es negativo, se suma 26 antes de aplicar el módulo $((c - 'A' - \text{shift} + 26) \% 26)$, garantizando que el resultado siempre caiga dentro del rango válido del alfabeto (0-25).

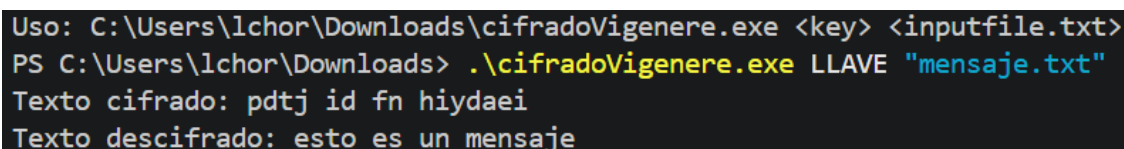
1.5 Salida de Resultados

```
printf("Texto cifrado: ");  
// ... impresión carácter por carácter de encryptedText  
printf("\n");  
  
printf("Texto descifrado: ");  
// ... impresión carácter por carácter del texto descifrado  
printf("\n");  
  
free(encryptedText);  
return 0;
```

Listing 5: Salida de resultados del programa y liberación de memoria.

El programa concluye imprimiendo el mensaje cifrado y el mensaje descifrado, recorriendo cada buffer carácter por carácter con putchar hasta llegar al carácter nulo. Al final se libera con free la memoria reservada dinámicamente para encryptedText, ya que esa memoria se pidió manualmente con malloc y el sistema operativo no la recupera automáticamente al terminar el bloque; omitir esta línea produciría una fuga de memoria (memory leak).

1.6 Salida en Terminal



```
Uso: C:\Users\lchor\Downloads\cifradoVigenere.exe <key> <inputfile.txt>  
PS C:\Users\lchor\Downloads> .\cifradoVigenere.exe LLAVE "mensaje.txt"  
Texto cifrado: pdtj id fn hiydaei  
Texto descifrado: esto es un mensaje
```

Figure 1: Ejemplo de compilación y ejecución en terminal.

1.7 Repositorio y Código Fuente

Código fuente:

<https://github.com/HansMarcus01/Criptografia-2027-1/blob/main/cifradoVigenere/cifradoVigenere.c>

Link del repositorio: <https://github.com/HansMarcus01/Criptografia-2027-1>